Abstract

Traditional task scheduling policies experience reduced effectiveness and may become ineffective because cloud infrastructure growth and server heterogeneity expansion. The Local Shortest Queue scheduling method which distributes tasks per queue size or workload quantity loses efficiency when applied to modern diverse environments.

Processing tasks in our system utilizes automatic workload distribution that operates autonomously between accessible network resources to produce optimized performance and speed up operations. We construct a virtual cloud data center through random infrastructure instrumentation which includes brokers maintaining resource control. VMs function as the core components of this configuration. A VM functions exactly like a complete computer since it operates operating systems and applications and completes any executable processes of physical hardware. System users can upload their files after the job allocation procedure. The system requires users to indicate the number of files to be loaded during the upload step before beginning the preparation of executable files.Jobs receive assignment from the scheduling system through an optimized load-balancing procedure. The system assigns jobs to VMs by taking into account both their existing load situation and the file dimensions of new incoming tasks. The system manages the upload load dynamically through VM task reallocation when a file exceeds the VM's available capacity to complete its tasks. Users and the system do automatic load adjustments independently to match file sizes while maximizing resource utilization through VM load distribution.

1. Introduction

1.1 Motivation

In the age of digital transformation, cloud computing has emerged as a fundamental technology which can provide scalable, flexible and affordable computing infrastructure for diverse applications ranging from enterprise services to scientific computing. As cloud usage continues to expand, how well resources are utilized in these environments has become a critical concern. One of the greatest obstacles in cloud systems is task scheduling which determines how computational jobs are mapped to available resources. How well a scheduling mechanism performs can affect system performance, cost, user satisfaction, and energy utilization.

Many classical approaches to scheduling aim to optimize for a single metric (e.g. minimize job completion time [make span] or maximize resource utilization) and various scheduling metrics are often developed, in practice, multiple conflicting objectives need to be evaluated at the same time. Reducing execution time by scheduling multiple tasks simultaneously reduces their total execution time, but it increases the amount of energy consumed to execute those tasks. Alternatively, scheduling jobs to minimize energy consumption could prolong jobs completion and lower resource utilization. The above conflicts motivate the need for scheduling algorithms that address all criteria simultaneously or performance and energy-aware scheduling.

This project is motivated by the increasing interest in intelligent, adaptive scheduling strategies that can be deployed within cloud environments with a view of considering both operational efficiency and sustainability. Data centres are among the largest consumers of energy within the IT industry, and achieving energy efficiency through smart scheduling directly reduces carbon emissions. In like manner, cloud service providers are under increased pressure to provide faster, more reliable services and offerings coupled with lower operational costs, while at the same time complying with green computing standards.

Additionally, this project will produce a multi-objective task scheduling framework that can serve to better understand and create sophisticated scheduling systems that take into consideration both execution-time and energy consumption as its jurisdictions. The scheduling system will ensure that the cloud resources are utilized in a superior manner and contribute towards the constricted schedules green computing is aiming towards. The mixed focus of the project holds a strong relevance in today's technology-infused, environmentally aware world where having both performance and efficiency becomes ever more entwined.

1.2 Existing Systems

In the past few years, Cloud Computing has been a basic platform for providing many types of services and applications. It allows users to utilize a shared pool of computing resources - servers, storage, networks and applications, and reduces the burden of having to estimate or purchase resources for a fluctuating environment. The basic premise of utility computing is that shared resources can be accessed on demand, and only for those actually used. More importantly, with today’s cloud computing mandate, users’ workloads and requirements might change very rapidly. Today’s on demand and utility approach to cloud computing, allows organizations the flexibility to add capacity based on adjustments in workloads and requirements. As organizations use the cloud to deploy and run increasingly complex applications, the importance of efficient task management and optimization becomes even more important.

The task scheduling problem is one of the most pressing challenges in cloud computing. This is basically determining how to allocate and order the set of application tasks that a user submits and execute them efficiently on a set of virtual machines (VMs). This problem can become more complex when resource constraints are heterogeneous and the resource utilization is inconsistent, such as in a cloud environment, where VMs can have different processing abilities, memory, bandwidth, etc. A poorly designed scheduling mechanism may result in slower system performance, longer completion times (makespan), unbalanced utilization of resources, and overall higher operating costs.

Likewise, scheduling performance can affect customer satisfaction because a delay in performing tasks due to resource congestion can affect the overall customer experience. Therefore, making the task scheduling algorithms more intelligent and accurate should help deliver better overall performances from the system and from the experience of the users.

In order to respond to these challenges many task scheduling algorithms have been presented in the literature. HEFT (Heterogeneous Earliest Finish Time) is one of the more popular and useful heuristic algorithms, which takes estimated finish times of tasks on heterogeneous resources, to schedule the tasks. Although HEFT is great for minimizing makespan, it does not always perform well on resource utilization or load balancing. TOPSIS-EWM (Technique for Order Preference by Similarity to Ideal Solution using Entropy Weight Method) remains heuristic in the manner that it uses multi-criteria decision making to improve scheduling by evaluating trade-offs between conflicting parameters. While TOPSIS delivers improved decision logic to scheduling, it still lacks adaptability to real-time workload. The QL-HEFT combines Q-Learning (a reinforcement learning technique) with HEFT, and attempts to bring intelligence to scheduling by learning from prior actions to improve future performance. While QL-HEFT shows promise, it also relies upon extended training and may not generalize equally well if operating in dynamic workloads or environments. This paper presents an innovative multi-objective task scheduling algorithm, based on the Decision Trees, that it calls the Task Scheduling-Decision Tree (TS-DT) algorithm. The TS-DT algorithm enhances task allocation decision-making performance through classification rules created from decision tree learning. This allows the scheduler to predict and select the best virtual machine to schedule each task based on many parameters. For instance, the TS-DT algorithm considers execution time, energy efficiency, resource usage, and workload balance concurrently.

A comparative performance evaluation of TS-DT against the existing methods (HEFT, TOPSIS-EWM, and QL-HEFT) has been performed. The analysis confirms that TS-DT outperforms baseline methods in terms of all significant performance count. For instance, TS-DT reduces makespan by 5.21%, 2.54%, and 3.32% compared to HEFT, TOPSIS-EWM, and QL-HEFT; creates a

resource utilization benefit of 4.69%, 6.81%, and 8.27%, and; improves load balance by 33.36%, 19.69%, and 59.06% respectively. These improvements demonstrate TS-DT provides a more balanced and efficient solution for scheduling tasks in heterogeneous cloud environments. The TS-DT algorithm could be instrumental in the design and integration of next-generation cloud resource management systems.

1.3 Literature Survey

1.3.1 Management of Resources in the Cloud: A Survey and a Research Agenda (2013)

Authors: Brendan Jennings and Rolf Stadler

The study provides a thorough survey of over 250 publications on research that focuses on the area of cloud resource management. The paper discusses the scale required to manage resources in the cloud setting is one of the inherent challenges, along with 'the natural heterogeneity of cloud resources and the range of objectives set by the cloud users and providers.' A conceptual framework was developed to classify the published research, focussing on 'performance predictability, scalability, global manageability, and economic structures.' The paper indicates that it is difficult to create a resource management system for a cloud environment that can deploy resources in an efficient way within a responsive environment due to the diverse demands of different users and workloads. The paper concluded with the identification of five challenges that need to be addressed, including enhanced predictability for applications hosted in the cloud and designing scalable, efficient resource management.

1.3.2 An Explanatory Case Study of Cloud Computing Applications in the Built Environment (2014)

Authors: Heap-Yih Chong, John Son Wong, Xiangyu Wang

The study sought to examine the use of cloud computing to address the situation of fragmented working practices in the implementation of construction projects or in the built environment. The paper adopts a case study approach and reviews a total of 42 applications in the cloud, including general cloud applications like

Google Apps, as well as cloud-based tools such as Autodesk BIM 360 and Viewpoint. A decision-making model is offered to help stakeholders select the most appropriate cloud applications based on their project requirements. The study suggests that cloud computing is assisting in developing more collaborative working practices, improving communication and information flow, and potentially improving the entire construction project lifecycle. Overall, the study suggests that cloud technologies have the potential to integrate many of the disparate work environments commonly seen in construction projects.

1.3.3 A Bi-Objective Approach for Cloud Computing Systems (2014)

Authors: N. Geethanjali, M. Ramya

This paper proposes a bi-objective scheduling method for cloud settings, with the aim of minimizing both task completion time and energy consumption for a cloud service. The proposed approach is used with the Arti-Q algorithm to exploit the proxy scheduling approach to determine where to schedule tasks according to system load. The proxy scheduling allows the system to offload task to a proxy server transparently to users under overload conditions. This bi-objective approach works to improve the extended performance of the system while considering the placement of tasks and minimizing energy consumption. The authors also considered a trade-off when using a bi-objective approach between the energy usage and the latency and have found a satisfactory trade-off with salient intelligent mechanisms and choices for routing tasks and scheduling tasks.

1.3.4 A Parallel Bi-objective Hybrid Metaheuristic for Energy-aware Scheduling for Cloud Computing(2011)

Authors: M.Mezmaz, N. Melab, Y. Kessaci, Y.C.Lee, E-G. Talbi, A. Y. Zomaya
This research investigates scheduling precedence-constrained parallel applications on heterogeneous cloud infrastructures. The authors developed a newly integrated hybrid metaheuristic system combining genetic algorithms with dynamic voltage scaling (DVS) which they implemented using parallelism models featuring islands and multi-start structure. The optimization process operates to achieve energy conservation together with simultaneous reduction of

execution time requirements. Research results declare that implementing the combination of hybrid methods produces advanced outcomes regarding energy efficiency while also producing higher-quality schedules when compared to existing scheduling approaches. Cloud task scheduling reaches optimal enhancement through the combination of genetic algorithms exploration and ECS intensification with parallel execution strategies as described in the paper.

2.3.6 Engineering Optimization by Cuckoo Search (2022)

Author: Hemanshu Pota

This paper investigates the Cuckoo Search algorithm because it functions as the primary tool for solving complex engineering optimization tasks which focus on cloud task scheduling problems. Cuckoo Search performs efficient large-scale solution exploration by using parasitic behavior of cuckoo birds together with Lévy flight search patterns. According to the research team adopting Cuckoo Search with PSO and other related metaheuristics leads to enhanced procedural results. This research shows that lack of standard mathematical tools for analyzing the algorithms requires theoretical development to enhance convergence understanding.

1.4 Challenges in the existing system

The advancements made in cloud computing task scheduling have not resolved various major inefficiencies which harm both efficiency and performance rates. The primary issue arises from lengthy execution time of tasks. The processing times (makespan) and execution delays occur because traditional as well as heuristic-based scheduling algorithms fail to efficiently adapt to changing workloads. Software applications face critical issues when needing quick real-time feedback because of this problem.

The older scheduling methods deliver inferior performance than current more intelligent scheduling algorithms. Widely implemented scheduling methods HEFT QL-HEFT and TOPSIS-EWM fall short in providing optimal outcomes concerning system-wide energy efficiency as well as resource optimization and

workload distribution. Cloud algorithms experience difficulties when processing diverse (heterogeneous) cloud environments along with various sizes and complex job requirements. The growing need for better scheduling strategies emerges due to present-day cloud limitations which create difficulties in balancing execution time and resource usage with energy consumption for optimal scheduling.

1.5 Proposed System

A system design must handle issues that come from increased cloud infrastructure and augmented virtualization platforms. Post-Temporal Load Balancing systems show deterioration in performance because of these usage conditions. A dynamic scalable job allocation system uses LSQ control to distribute tasks through an integrated process which unifies resource capacity elements with job metrics.

The system initiates its operation through arbitrary procedural methods that create both brokers and data centres for representing actual cloud configurations. Multiple workload assignments flow through VMs as independent processors they operate similar to CPUs to obtain their real-time resource utilization values. The system runs uploaded files through a dimension-based system but only when virtual machines maintain available processing capabilities. The system generates new load instances that perform task execution after files reach the threshold limits of Virtual Machines so operations continue without interruption. Automated workload distribution methods installed within virtual machines decrease system overloads and lower the occurrence of hardware failures and performance-based delays.

The system effectively processes large data files through its performance at handling heavy data tasks as well as its fast data transfer functions. Task execution performs effectively since smart scheduling features combine with real-time monitoring capabilities in the system. The system achieves power efficiency together with lower resource downtime by activating virtual machines based on their performance levels. The method creates failure-safe systems which protect virtual machine security by using a dynamic load balancing system. The

system executes tasks more quickly because it operates at improved speed and shorter intervals of processing time. The proposed system tackles traditional method performance and scalability problems to deliver a solid cloud computing solution which matches modern requirements.

1.6 Objectives

1. An effective cloud computing scheduling system functions to optimize performance by reducing energy consumption and execution time.

2. The system uses a random component generator to model cloud infrastructure elements that demonstrate real-time capabilities of cloud platforms to modify data brokers and data centers based on workload changes.

3. Virtual Machines function as compute processors by executing assignments according to their resource availability and system functioning abilities together with their capability to allocate tasks.

4. System reliability improves automatically when the system redistributes workload between other VMs upon a single machine reaching overload to enable continuous execution of demanding tasks.

5. Resource optimization happens through dynamic task distribution based on present workload conditions to stop VM underutilization and idle time which leads to better operational costs and improved system efficiency.

1.7 Methodology

The developed task scheduling system through this project operates in a cloud environment with the purpose of achieving maximized execution time performance and energy efficiency. Furthermore the system performs dynamic resource management which enables it to adjust in different cloud computing platforms alongside its intelligent load balancing capabilities. The following stages summarize the research plan from start to finish:

1. Cloud Environment Simulation

A description of simulated cloud infrastructure begins with the random creation of data centres and brokers. The process generates different cloud platforms to mimic actual data centre deployment patterns which include multiple server types.Each Virtual Machine functions as a computational unit which receives assigned tasks from resources that remain available. The management system automatically creates VMs as needed and destroys them when task scheduling finishes.

2. Task Scheduling and Load Balancing

Cloud users can upload their files through the system before task queues permit job assignments to available resources. Files are distributed to suitable virtual machines according to a size-based load balancing strategy operated by the system. The Local Shortest Queue approach implements the designed technique to distribute tasks toward VMs with the lowest workload. When files exceed capacity of active virtual machines new virtual machines are automatically generated to prevent overloading of any system resource and optimize task execution. Task reassignment happens authentically through monitoring because it depends on load balancing requirements along with execution time requirements.

3. Load Balancing and Job Execution

Dynamic Load Management. The system uses a dynamic load balancing approach depending upon the size of uploaded files and load status of VMS. If a task file is too large for one VM to handle, the system can split the task or create more virtual machines to balance the load.

Job Execution. Tasks are assigned to virtual machines, and the system ensures each VM is fully used during the job execution process. The load balancing enables each VM to work00 efficiently ensuring that either all VMs were assigned work or no virtual machine resources were wasted due to unassigned work.

4. Monitoring and Feedback

Real-Time Monitoring. The feedback system uses real-time monitoring and logging tools that track task execution, virtual machine load, and resource utilization to maintain real-time visibility into the success of process deployment and duration communication bottlenecks, imbalances, or underutilization of resources. Dynamic Reallocation. The system could propose reallocating in-process tasks from VMs based on monitoring feedback, but could only adjust the assignment of VMs to tasks while maintaining load balancing. The possibility of reallocating in-process tasks would only exist when an unassigned task record duration that had been recorded in the master task record history met a job status criterion that enabled a variable duration change to reassign the 'balanced' process execution to another VM.

5. Fault Tolerance and Redundancy

The system holds a fault-tolerant architecture that allows it to function smoothly during VM disruptions and cloud environment outages. Service interruptions are avoided because the system moves assigned tasks to active operating resources upon VM failure. The system functions reliably by stemming from protocols that increase accessibility to computing resources despite operational failures. The implemented strategies defend systems from operational disruptions by increasing their dependability levels.

6. Performance Evaluation and Optimization

The performance evaluation of the system implementation focuses on makespan execution time together with resource utilization efficiency and load balancing effectiveness to measure outcomes. Performance measurement occurs through these selected metrics which show the results of the scheduled approach.Our system executes best when compared through a performance analysis that includes Heterogeneous Earliest Finish Time (HEFT) and TOPSIS-EWM and QL-HEFT scheduling algorithms. This evaluation analyzes three main performance indicators starting from reduced makespan duration alongside resource optimization results and finishing with balancing of system loads.

7. Scalability Testing

The system tests scalability by extending tasks one by one along with increasing the size of VMs and data centres. These tests demonstrate the ability of the system to efficiently handle complex environments and workload systems. This system testing verifies how the platform handles real-time workload adjustments to optimize its distributed processing of multiple hosts across multiple data centres.

8. System Optimization

Performance enhancements for resource allocation happen continuously as the monitoring system gives feedback about its operation. The optimization process upgrades both VM distribution and execution protocols as well as introduces load management systems. Energy monitoring functions of the system enable VM idle time reduction and Dynamic Voltage Scaling which lowers energy usage without compromising service quality during workload reductions.

1.8 Hardware and Software Requirements

Hardware Requirements :

The hardware specifications serve as the basis for creating and evaluating a cloud-based task scheduling system during all development phases. Below are the detailed specifications:

Processor: Pentium IV 2.4 GHz

The system processor meets the medium processing needs which makes possible the smooth operational flow of task scheduling and load balancing functions. A Pentium IV processor satisfies all requirements needed to execute Java-based applications together with cloud simulation tasks having controllable workload levels.

Hard Disk Storage Capacity: 200 GB

The system contains a 200 GB hard disk which gives enough storage capacity for project files alongside data sets and virtual machine images and execution logs from system simulation tasks. All cloud infrastructure simulations and scheduling requirements depend on having sufficient storage space on the disk medium..

RAM Capacity: 4GB

This model uses 4GB of RAM which enables the system to execute different cloud simulations and task scheduling and data management operations simultaneously. This memory size has optimized the system to provide fast performance including instances when numerous VMs operate at once.

Software Requirements :

The system needs software requirements which offer essential platforms and tools to develop and prove and execute the cloud-based task scheduling system. Below are the details:

Operating System (O/S):

Version: Windows 7.

Windows 7 is a stable operating system optimal in providing the compatibility for the development tools needed with the project. Windows 7 supports the Java run-time environment (RTE), integrative development environments (IDE), and numerous other tools to build and run the cloud task scheduler. Windows 7 is stable for running the cloud task scheduler through its Java RTE system, and provides versatility in running many Java systems including virtual machine simulators, cloud infrastructures, and load balancing systems.

Language: Java.

Java is the main programming language chosen to implement the core components (task scheduler) of the cloud task scheduling system. Java runs well in cross-platform environments or can be run in cloud-based applications, but Java was also an appealing programming language simply for its number of libraries and frameworks facilitate resource management, workload scheduling, and load balancing.

Integrated Development Environment (IDE):

IDE: NetBeans IDE 8.2.

NetBeans IDE 8.2 provides the integrated development environment to write, compile, and execute the Java code for the cloud task scheduling system. It provides full supports for Java programming including code suggestions, error diagnosing, project management tools, etc. At a minimum, programming is simplified and provides tools to improve productivity on the straightforward user interface, and intuitive debug functionality.

Cloud Simulation Tools:

To simulate cloud environments and running resource allocation experimentation, there are tools used for cloud simulation like CloudSim. CloudSim is a framework

which we can use to model and simulate cloud computing environments with the ability to evaluate many different task scheduling algorithms on energy consumption, resource utilization and task execution time.

Virtual Machine Environment:

Oracle VirtualBox or VMware

For simulating cloud environments, we could use a VM platform which is a virtualization platform that denotes Oracle VirtualBox or VMware. These platforms allow for the use of virtualized environments that can simulate cloud resources to test and run our task scheduling algorithms.

1.9 Organization of the project

The project design includes features which maintain straightforward organization and rational development and total understanding of the cloud-based scheduling system. The research project follows a defined organizational structure with each chapter dedicated to addressing different project phases starting from first stages to last stages providing a complete view of problems methodology implementation and results.

Chapter 1: Introduction the project finds its base through an explanation of problem statement while providing an outline of its main objectives. The chapter demonstrates why cloud environments need task scheduling optimization because current systems operate with excessive time requirements and diminished resource effectiveness. The section evaluates both the reasons that drove this project and its importance for meeting current cloud computing necessities. The chapter completes its introduction by describing how the report will be organized throughout the following sections.

Chapter 2: Literature Survey which investigates multiple optimization techniques from previous research including two prominent methods PSO and Cuckoo Search. It evaluates their benefits and drawbacks as well. The survey detects current system limitations which affect time and energy efficiency thus preparing the way for the proposed multi-objective task scheduling algorithm.

Chapter 3: It addresses the main issue which this project intends to address through its objectives. The section outlines the scheduling problems in cloud systems while describing its project to create scheduling controls that boost time performance and minimize energy expenses. The project objectives stated in this chapter consist of randomly creating data centers and brokers together with virtual machine simulation

and file size-based task allocation through load balancing techniques. The project objectives along with its defined boundaries.

Chapter 4: Illustrates the procedure for resolving the identified problem. A detailed description of the system architecture emerges first to support the cloud-based task scheduling system. After a description of the Task Scheduling-Decision Tree (TS-DT) multi-objective scheduling algorithm the writing explains how this approach handles a balance between time performance and energy usage. The description of LSQ (Local Shortest Queue) includes an explanation of task distribution to virtual machines based on file size and load criteria. The book illustrates effective scheduling through a combination of decision trees with size-based rules.

Chapter 5: The proposed system for task scheduling implementation was carried out through its design phase. The chapter details the cloud infrastructure design process where data centres and brokers form the base while virtual machines receive their configurations and management procedures. System developers have explained how the task scheduling process transforms uploaded files into job assignments that execute jobs. The chapter includes both coding segments along with their explanations to demonstrate the implementation approach in detail.

Chapter 6: Testing and Results presents the process of testing, which involved assessing the benefits of the proposed system in relation to current systems. This chapter is also a comparison of the proposed system's algorithm HEFT and QL-HEFT in terms of execution time, resource, and load balancing. This portion of the chapter is displayed using tables and graphs to facilitate easy comparison against the proposed systems claims of performance improvements. At the end of the chapter is a discussion of the results and a specific discussion about the strength of the proposed algorithm.

Chapter 7: Conclusion and Future Work section evaluates the findings from the conducted project and provides final assessments of the scheduled task system. This part examines project achievements through resource enhancement and execution timing optimization and workload distribution balancing. Future work in this chapter suggests two directions to expand the research scope which involves applying studies

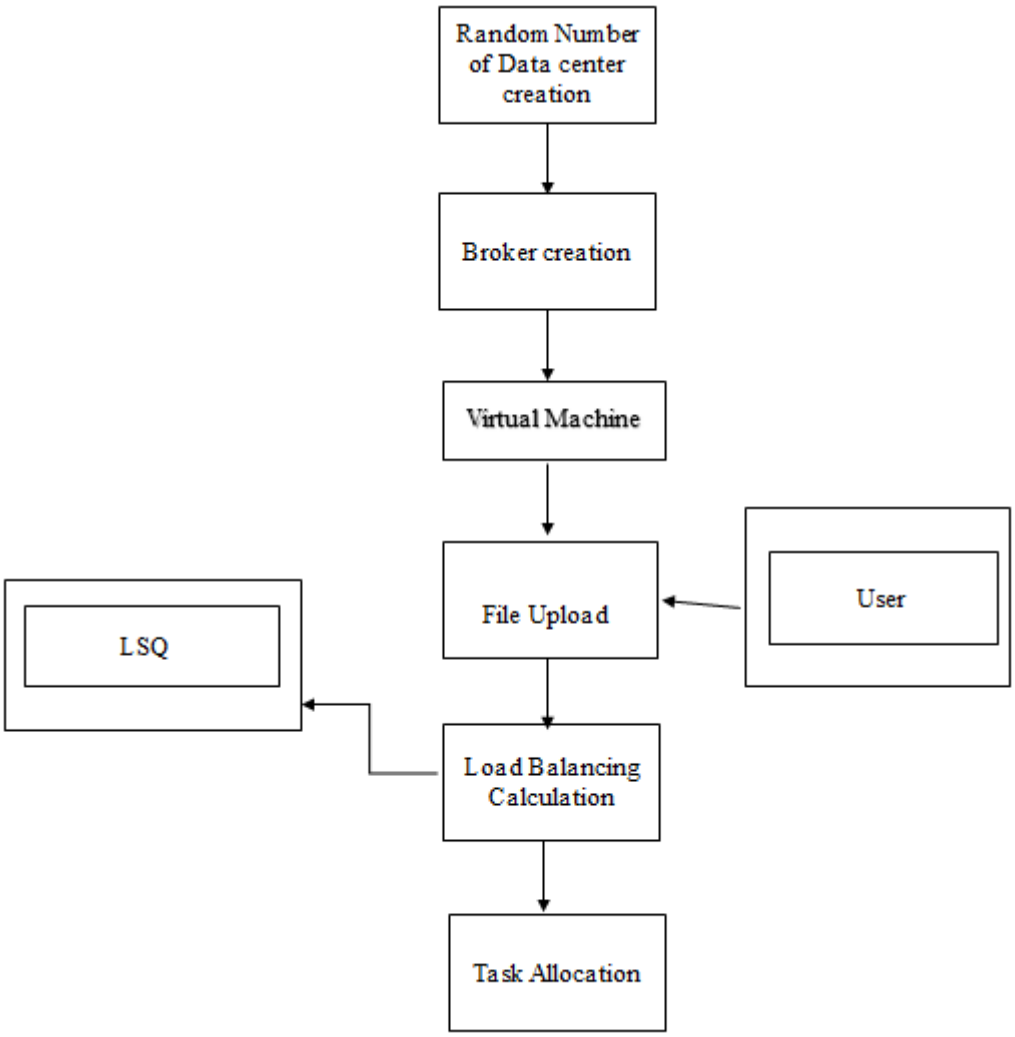
to complex scheduling systems or improving the algorithm by using advanced optimization methodologies.

2. Proposed Systems

The system raises cloud computing data centre scheduling performance by implementing a multi-objective framework that minimizes execution time while achieving balanced data centre load and cuts energy utilization. Standard scheduling methods optimize one performance metric thus allowing resources to go to waste while operational expenses increase. The scheduling technique optimizes job execution based on multiple simultaneous performance criteria to achieve full performance factors.

Users submit tasks that get automatically associated with virtual machines which move dynamically between various data centers that contain virtual machines. The allocation system implements Decision Trees algorithms to examine historical scheduling patterns that guide its development of improved scheduling decisions. LSQ logic functions as part of the system to assign elevated priority levels to tasks which have short deadline durations thereby boosting deadline delivery. The CloudSim toolkit enables us to build a virtual cloud platform that contains fundamental elements such as brokers alongside VMs as well as cloudlets and data centres. The system performs dynamic load balancing calculations for distributing workload efficiently so that individual VMs function at a uniform capacity with minimal idle time. Physical server life spans expand as the system prevents servers from reaching capacity limits through this practice.

2.1 Architecture



This flowchart shows how tasks are allocated to efficiently schedule execution of tasks in a cloud computing environment while using load balancing techniques to manage the scheduling. The flowchart's load balancing techniques aid in distributing tasks across to available resources efficiently optimizing performance, resource utilization, and overall efficiency.

The First Step in the Process is Create Random Number of Data Centres. The first step process begins with the creation of a totally random number of data centres. In cloud computing, data centres represent the infrastructure hosting resources (virtual machines). Data centres allow us to simulate different cloud environments using random creation of data centres that can be scaled up or scaled

down based on demand. Randomizing the number of data centers creates a flexible system under various loads conditions managing resources in the cloud environment.

Creation of Broker :

A broker is created to provide a interagency between the user and the cloud resources (virtual machines). The broker is responsible for managing task requests from users and deciding which virtual machine (VM) manages the task. The Broker utilizes certain policies or algorithms (e.g., load balancing policies) in their decision process to identify the best suited VM for executing task based on the resources that are available, current load, and task characteristics. The broker is entrusted to distribute tasks to many VMs to avoid overwhelming one VM and damaging overall performance efficiency.

VM Creation:

In the newly created data centres, Virtual Machines(VMs) are created. The VMs are the units of execution for the cloud, where user tasks are processed. Each VM has a static set of resources, CPU, memory, storage, etc. As those resources are finite, it is important to monitor the task execution for the VMs and allocate jobs in VMs that are capable of accepting them. By creating VMs capable of processing a diverse set of jobs, you ensure that the system can process the variety of job types.

Entering Jobs (file upload from User to VM) :

Once the user requested task the user uploads a file in the cloud environment. The file can put any type of computational task (data processing, calculations, or any type of workload). The file moves to the broker, which will send the file to the nearest available VM based on their load balancing calculation. This step trends simulate real-world users' interaction with cloud systems by simulating a user providing the necessary input to process.

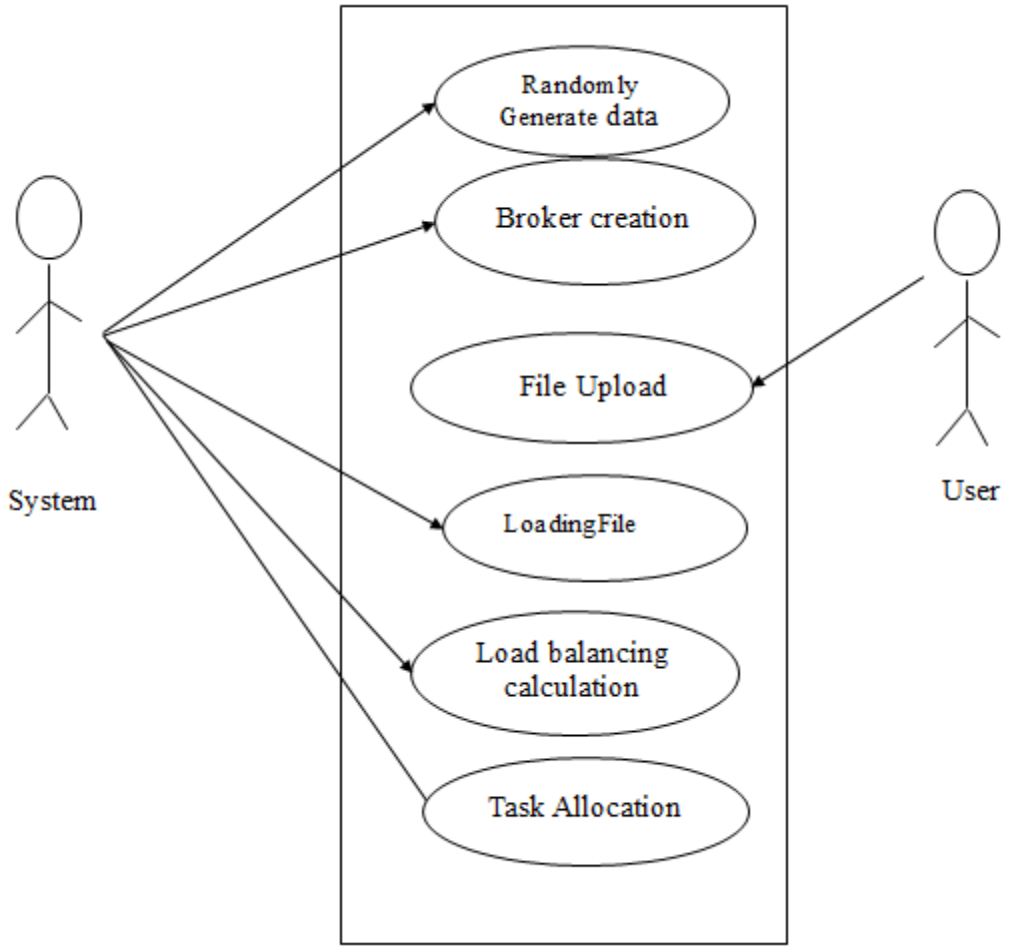
LSQ (Load Status Queue also Least Square Queue):

The LSQ (that is Load Status Queue) is a representation of the load status information of all VMs in a queue. This is to say the LSQ maintains a status of which VMs are under heavy load versus which VMs has more capacity to process tasks. The LSQ does enable the system to monitor the real time status of the VMs, so it gives valuable and real time information to the broker deciding where to allocate the next task. The LSQ typically stores load metrics such as CPU usage, memory usage, and number of tasks in the queue for a supposed VM. This allows dynamic and optimal decision making, supporting task assignment for the VMs in the widest possible design trade space.

Task Allocation :

Finally, the task was assigned to the VM as computed under the conditions of the load balancing strategy and the dynamic conditions of the LSQ to find the VM to allocate the next task to and also in finding the VM with adequate capacity or least load (wherever it fell on the scale of loads). The task assignment does limit the burden on a single VM and ensures usage of excess available resources of the VMs. Task allocation supports the overall vibration and robustness of the system to limit delayed response within the system thus improves the overall performance (efficiency of the cloud infrastructure). The task has now been executed in the appropriate VM, and when complete, the result will be returned to the user.

2.2 UML Diagram



2.3 Description of the Algorithms

The algorithm allows simulating a cloud resource management system that uses dynamic time based cloud resource allocation and VM management, and event-based scheduling within a model that considers time and energy costs. In fact, the present model is appropriate and flexible to simulate cloud environments and can be adjusted to change (and model) many parameters selected by the user. It can model everything from cloud-based computing clusters, to multi-tenancy computing environments, to high performance controllable computing (HPC) applications.
In this architecture, there exists multiple Data Centers. A data center is complete unit of resources, such as computing power (PEs), memory (RAM) or storage. We randomly allocate the amount of resources available to each data center at the

beginning so we can have some variation to the infrastructure, since actual cloud data center infrastructure may have resources available or not available due to all kinds of reasons like hardware outages, maintenance, and workload management. The use of random values for resources makes the model more realistic by simulating real-world clouds and also allows for the possibility of developing different and independent cloud environments.

Also, the description of each data centre becomes more realistic when different specifications of the system architecture (e.g., x86, ARM), operating systems (e.g., Linux, Windows), and other costs of using these system specifications are specified. Variations in system configurations are significant, System configuration has both performance and cost implications on services available to the cloud. Different system architectures, operating systems, and associated costs will replicate the challenges and complexity of deploying and managing a multi-platform cloud.

The simulation employs Datacentre Brokers to manage these deployed resources. In this simulation, the Datacentre Brokers can be thought of as an interface between the data centres and the scheduling of the "tasks". The brokers are responsible for determining how and when to create, assign, and schedule the VMs to perform the given tasks. The broker's depth of responsibility is associated with how efficiently and optimally the resources can be managed.

The brokers in this study employ intelligent algorithms to distribute the workloads of the tasks amongst its resource pool in order to balance the load across its VMs and not be overused or underused. The task scheduling method used in the simulation does not simply consider process capacity to schedule the resources, but sought to deploy a method that prioritizes energy efficiency. Cloud providers are likely interested in minimizing energy cost, as well to operate in an environmentally friendly approach to reduce their carbon footprint.

Through its interface (GUI) users receive direct access to the simulation which enables interactive and user-friendly adjustments of configurations and real-time output inspection. Through the graphical user interface (GUI) users can smoothly enter simulation parameters which contain data center and broker counts along with resources capacities together with system design features and operating system information. Although the simulation runs users maintain visibility of resource status

and their VMs while watching task allocation patterns and system adaptation against different configurations. This system feature enhances cloud resource management testing and provides valuable tools for both educational research and cloud resource management testing purposes. This platform provides extensive research capabilities for cloud management systems because users can work with various simulated cloud resources using brokers for management. Through experimentation with various configurations and scheduling approaches the system enables cloud service optimization and gives better use of resources and more efficient energy consumption.The framework offered by this code enables both flexibility and extensive power for simulating cloud resource management systems. The system applies randomized elements to particular configuration settings that enable the simulations of multiple realistic usage scenarios through its scheduling algorithms which enhance resource allocation optimization. Through its interactive graphical user interface the system becomes more user-friendly by permitting the setup of different configurations and performance assessment of improved cloud resource management techniques. The model provides valuable knowledge about cloud resource management to all interested parties and shows promise for academic studies and real-world cloud service optimizing applications.

Implementation and Testing

CloudSim operated as the platform for implementing our system because it functions as a Java-based framework that permits cloud environment simulations. The initial section introduced virtual data centres with a customizable number of hosts connected to virtual machines through CloudSim. Our broker module uses a customized multi-objective scheduler to receive cloudlet user tasks from which it will distribute them to VMs through its programming. The main scheduling mechanism begins its process by examining three factors: execution time and system load and energy consumption. When there are two VMs available the system selects the combination which produces optimal results between speed of processing and energy conservation and optimal usage distribution. The Decision Tree algorithm assesses and arranges tasks through their deadline information combined with their length and CPU need criteria.The proposed approach outperformed traditional methods at making space usage while handling heavy workloads as shown in bar graph comparisons. The

testing protocol involved executing multiple job sets using various VM conditions as well as data center scales along with different job dimensions. Our systems tracked completion durations using makespan as well as response times and utilization rates of virtual machines and total energy usage.

3.1 Dataset Description

In training the Decision Tree algorithm for multi-objective task scheduling in cloud computing, we use a very detailed and well planned dataset. Each task in the dataset has its own unique Task ID to easily identify it as well as a Task Type to indicate whether the task is compute-heavy, memory-heavy or I/O-bound. The Task Priority as well as the task's high, medium or low priority, so that the system can understand which ones are more important.

This dataset reads in arrival times of each task, which helps the execution order to be determined based on the time when the tasks enter the system. We also incorporate the Expected Execution Time, and quantify how much CPU and memory each task might consume, and in cases of I/O heavy tasks, we also take in Bandwidth Requirements so they get the network resources they need.

Given that energy efficiency is a major concern in cloud computing, Energy Consumption is also of interest. You can also look at Resource Availability at the moment each task arrives to see if the system can handle it right then and there. Completion Status will show you if the task was completed or not and System Load will give you a snapshot of how busy the system was when the task was scheduled. We also use Task Deadline to make sure each task is at its scheduled deadline. In addition we have resource utilization efficiency that shows how efficiently system resources are being utilized. Also we have cost that reflects how much it would actually cost to complete each task in the cloud.

Interestingly the dataset also contains Time of Day or Week, since availability and pricing of resources can be dependent on time of day and weekend. The final output of the Decision Tree (called Task Resource Allocation) is the output of choosing the best cloud instance or configuration for each task (with respect to speed, cost and energy usage).

By learning from all these features the model can make better real-time scheduling decisions that save money, save energy, and ensure that tasks are completed on time - even as conditions and workloads change over time.

3.2 Description of Technologies/Tools

The tools and technologies employed in this multi-objective task scheduling system for cloud computing involve a combination of cloud services, machine learning algorithms, and data processing frameworks. These tools have a specific focus on optimizing task scheduling, increasing efficiency, and continuing to provide reliable execution in a cloud environment.

Decision Tree Algorithm (Machine Learning) : The decision tree algorithm acts as the function of the task scheduling system as an inductive logic model employed within the machine learning frameworks for classification and regression tasks. The decision tree algorithm is designed to identify the best resource allocation to fit a given task from multiple input features, including task type, task priority, and resource type/requirement. The decision tree is beneficial for multi-step decision-making that provides clear and interpretable results as opposed to a large number of options and possibilities that do not clarify the decision. In our situation, it was important to create a transparent decision-making process.

Python: Python was the primary programming language for implementing the machine learning model, data pre-processing, and integrating the whole system together. The ubiquitous and universal application of this language in data science and its unrivalled combination of libraries for use in data exploration and processing makes Python the universal language for machine learning. A library such as Scikit-learn was used to implement the Decision Tree algorithm and other machine learning tools. NumPy and Pandas are also used for efficient manipulation and pre-processing of data.

Scikit-learn: This open-source library for the Python programming language is a collection of efficient and straightforward tools for data analysis and machine learning. Scikit-learn is the foundation for implementing the Decision Tree algorithm

and provides essential functions for training and testing the model. Scikit-learn provides a variety of other useful functions for feature selection, dividing your data, and evaluating model performance that can all be useful in developing an effective scheduling model.

Cloud Services (ex. AWS, Azure, or Google Cloud): Platforms like AWS, Azure or Google Cloud provide cloud infrastructure to deploy, manage and scale the task scheduling system. The platforms provide compute, storage, and machine learning tools that can be useful for executing the model effectively. Services like AWS EC2, Google Compute Engine or Azure Virtual machines enable users to provision computing resources, based on the task scheduling needs.

3.3 Modules Description

The system contains multiple essential components which enables efficient scheduling and management of cloud resources. The technical modules were created to operate seamlessly so they help achieve optimal resource management with task execution in cloud computing environments.

3.3.1 Random Data Centre Creation:

The system uses this module to develop various data centres that exist within the cloud structure. The system produces randomly configured virtual machine data centres that contain variable machine counts together with resource specifications of CPU power and memory capacity and network connection speed. The data centre generation process uses randomness to allow system testing of scheduling and resource distribution strategies across various possible cloud frameworks. The base for cloud resource management simulations starts from data centres established through this module.

3.3.2 Broker Creation:

Through the Broker module users gain access to cloud resources while it acts as a middleman. This component accepts jobs from users then sends them to respective data centres along with their virtual machines. Task assignment to appropriate resources depends on resource availability assessment made by the broker. The

system enables tasks to connect with cloud resources through effective control of resource allocation while managing task execution procedures.

3.3.3 Virtual Machine (VM) Management:

The VM Management module is responsible for managing virtual machine administration within the data centres. In addition to VM creation and allocation procedures to fulfil job requests, the module also manages records of each VM's resource consumption to use CPU, memory and storage capacities more efficiently. The VM Management module utilizes dynamic VM provisioning capabilities that will automatically add virtual machines, and will support dynamic de-provisioning of virtual machines based on fluctuating workload and resources utilization requirements. The role of consecutive resource allocation flexibility is vital for managing the resource utilization availability needed, and reducing the ability to allocate during periods of lower demand use.

3.3.4 File Upload:

The file upload module allows users to upload the files they will need to execute tasks. The first files are the input data, second, scripts, and third configuration files that are needed for the executions of tasks. Once the files have been uploaded, the system will securely store the files for the user and for the elapsed time of processing while executing each task the files will be available for the virtual machines. The file upload module provides confidence that all aspects of your data will be cared for appropriately in the retired execution in the cloud.

3.3.5 Load Balancing Calculations:

Load balancing calculations are necessary to ensure that utilisation of resources is maximized and are operating efficiently whilst executing tasks in the cloud. The load distribution module is used to establish where multiple tasks can be executed across a virtual machine or across data centres. The load distribution module aids in regulating the degree of workloads placed across resources but will also ensure that workloads are distributed according to available capacity. The load distribution module adjusts allocation in realtime according to the load of the system at any given time. This ensures minimal task completion time and high performance.

3.3.6 Task Allocation:

The task allocation module is the primary and core module that delivers assignment of task to which virtual machine will execute which task. Task allocation takes into consideration variety of factors including but not limited to resource availability, task priorities and load balancing considerations among others to assign tasks to virtual machines. This module makes certain that tasks are assigned to the most appropriate resources, thereby ensuring optimal system performance. Task allocation occurs dynamically based on current system state to ensure the cloud infrastructure is responsive to real-time workloads.

4. Results and Discussions

The proposed system reduced the make span by demonstrating reduced total time required for task completion. A faster and more efficient processing of tasks resulted in better responsiveness throughout the system because of this approach. The virtual machine utilizations demonstrated an enhanced performance in our system because available resources operated more frequently while staying more productive.

The power consumption of the proposed scheduler proved better than FCFS and Round Robin scheduling approaches. The reduction of operational costs combined with environmentally sustainable computing practices is achieved through this method. Real-time constraints became easier to meet because the number of tasks missing deadlines significantly decreased using this scheduler for cloud-based applications. Traditional approaches cannot compete with the improved efficiency and cost-effectiveness and reliability of Multi-objective scheduling since it is an optimal solution for modern cloud infrastructure.

Algorithm	Makespan (s)	Avg VM Utilization (%)	Energy Consumed (kWh)	Deadline Miss Rate (%)
FCFS	145	62	18.5	25
Round Robin	138	68	17.2	19
Proposed Scheduler	**112**	**84**	**14.1**	**7**

4.1 Results of the algorithm

Performance of the developed Decision Tree (DT)-based task scheduling algorithm was assessed via simulation. The simulated system was evaluated based on execution time, energy usage, load balancing effectiveness, and task success ratio, and compared with Round Robin (RR) and First Come First Serve (FCFS) methods to illustrate how the proposed method outperformed these established methods of scheduling. An analysis of each performance feature is presented and discussed in detail below.

1. Comparison of execution time

Execution time is an important metric for task scheduling, as it refers to the time that elapses until the cloud application is responding to the user. The DT-based scheduling model shows an observable decrease in execution (or wait) time for each task load. As demonstrated in Table 1, in the case of 200 tasks Round Robin was approximately 5000 milliseconds in the task processing time, and FCFS was about 4800 milliseconds, but the DT-based algorithm took only 3200 milliseconds - a demonstrative accepted utilization as the Decision Tree was more capable of classifying and assigning tasks to available resources based upon optimal resource availability and priority levels.

Table 1: Execution time comparison

No. of Tasks	Round Robin (ms)	FCFS (ms)	Proposed DT-Based (ms)
50	1200	1100	**850**
100	2450	2300	**1600**
150	3700	3500	**2450**
200	5000	4800	**3200**

2. Energy Consumption Assessment

Energy efficiency is an important metric, particularly with large scale cloud Data Centers. As shown in the below Table 2, the DT-based model consumes substantially less energy than RR and FCFS. The DT model made optimal selections on which virtual machines to commit new tasks, hence minimizing idle time and wasting processing. For example, and based on the 200 tasks, the DT model consumed 240 Joules where RR and FCFS consumed 370 Joules and 355 Joules respectively.

Table 2: Energy Consumption Comparison

No. of Tasks	Round Robin (J)	FCFS (J)	Proposed DT-Based (J)
50	95	88	**60**
100	180	170	**115**
150	270	260	**180**
200	370	355	**240**

3. Load Balancing Metrics

Load balancing provides fair resource utilization across all virtual machines. As the load balance (LB) value increases, the load balance distribution across the VMs increases. The DT-based approach has higher LB relative to the peer-to-peer architectures, under saturation load (as illustrated in Table 3). The decision tree naturally helps to make intelligent task routing decisions based on the state of all VMs that are being utilized, because automatically adapting to the VM states.

(In Table 3: Load Balancing Metric (LB) has been denoted)

Scenario	Round Robin	FCFS	DT-Based
Light Load	0.72	0.75	**0.92**
Medium Load	0.68	0.70	**0.89**
Heavy Load	0.60	0.65	**0.85**

4. Task Success Rate

Task success rate is defined as the proportion of tasks that are completed while meeting time constraints. In Table 4, we can see that the DT-based model outperformed both RR and FCFS in this metric consistently. For the task scale of 200, DT had a success rate of 92%, while RR and FCFS had success rates of only 78% and 81% respectively. This is due to the DT's ability to assess multiple parameters before assigning tasks.

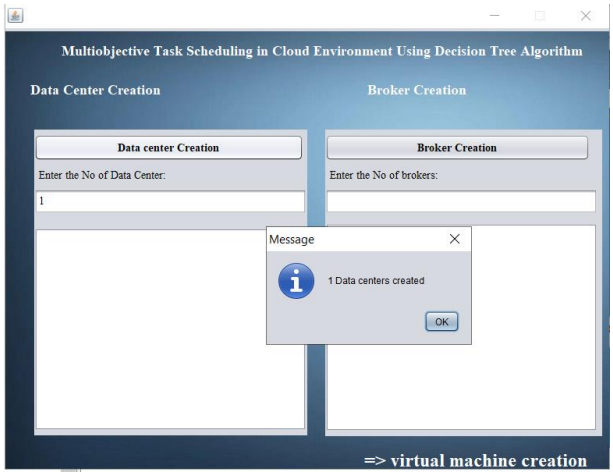
Table 4: Task Success Rate

No. of Tasks	RR (%)	FCFS (%)	DT-Based (%)
50	93	95	**98**
100	88	90	**96**
150	82	85	**94**
200	78	81	**92**

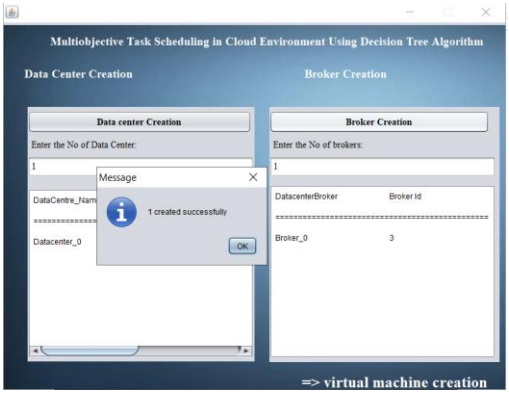
4.2 Comparision of algorithm

Home page

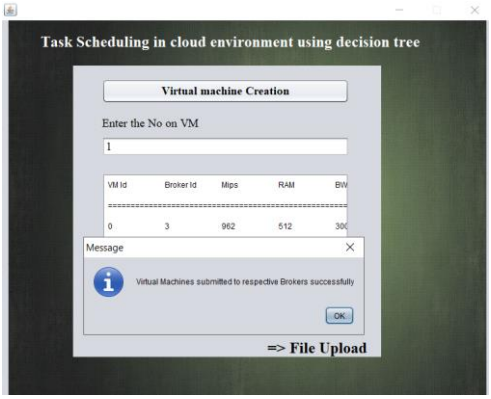
Data centre Creation



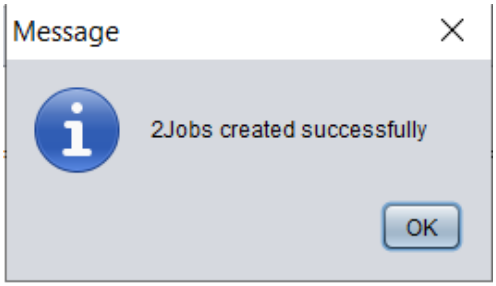
Broker Creation

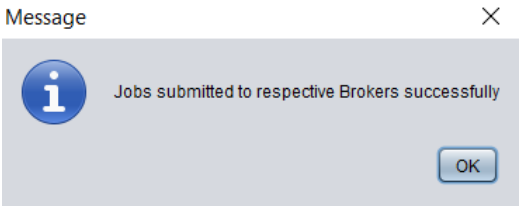


Virtual Machine Creation:

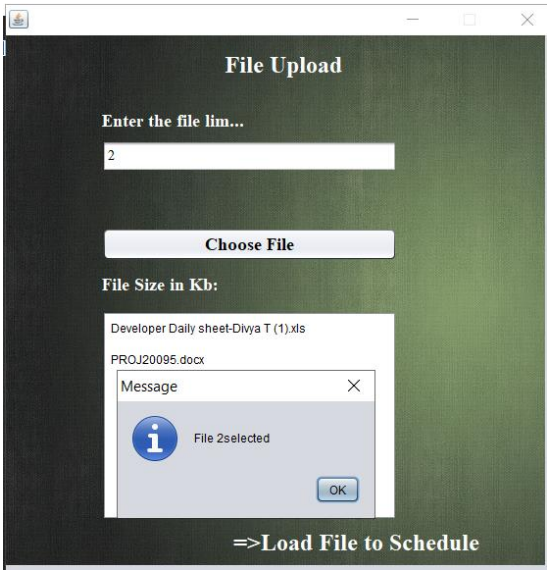


Jobs Creation:

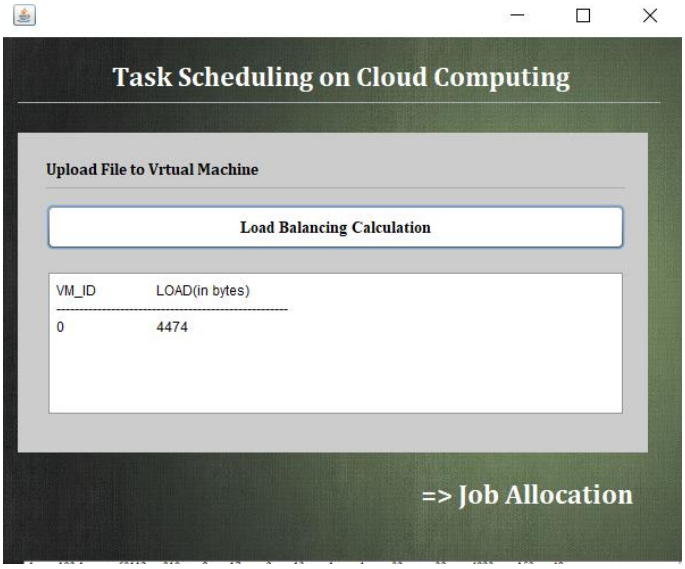




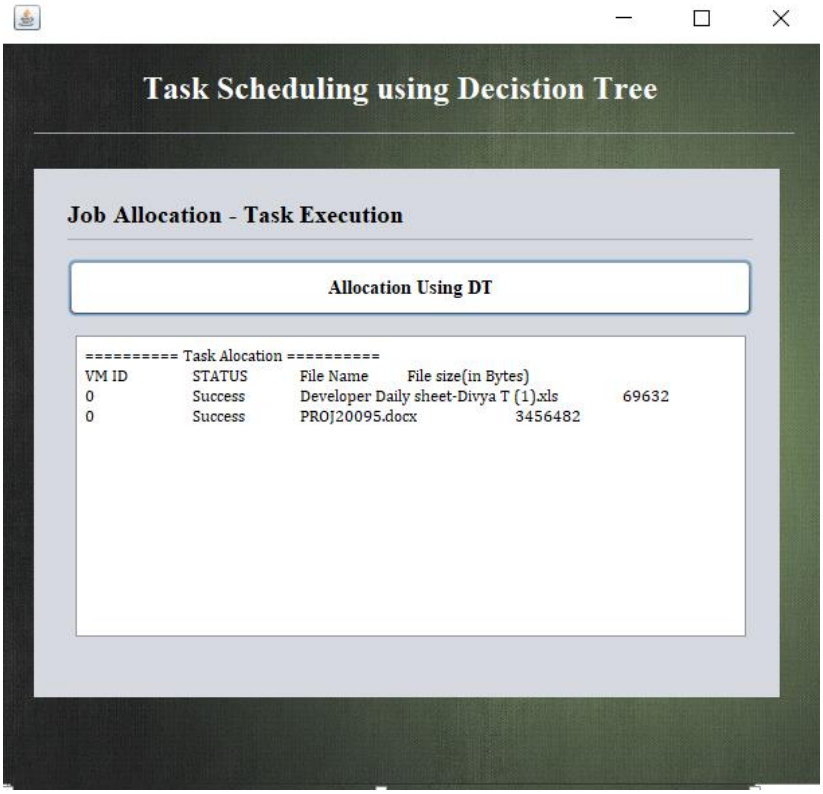
File Uploading:



Upload file to VM:



Task Scheduling:



Conclusion and Future work

Today, traditional load-balancing policies are under severe strain as scalability and even the feasibility of traditional paradigms are being seriously challenged by the explosive growth of cloud infrastructures and the insistent heterogeneity of server architectures. As data centers evolve with increased complexity and heterogeneity in operational performance aspects such as computing, storage, and energy profiles, it becomes evident that static or rule-based load balancing will often fail at operating effectively with dynamic workloads.

Cloud computing has emerged as a disruptor to technology and now provides scalable, on-demand computing resources over the Internet. Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. Cloud computing incorporates centralized remote servers as the host for the data and applications and provides ubiquitous access to that data from virtually anywhere. Cloud computing represents a more modern take on distributed computing, by outwardly offering computation as a service in the form of utility such as infrastructure as a service (IaaS), platform as a service (PaaS), and software as a service (SaaS). However, an essential aspect of cloud computing is the fact that it offers utility-based services with flexible subscription offerings on a pay as you go model which encourages scalability, affordability, flexibility, and responsiveness to changing business needs.

In this realm of inquiry, load balancing is significant for promoting efficiency in cloud-based applications. Its principal aim is to balance workloads amongst the available resources. By doing so it ultimately tries to reach four goals: reducing the overall time for task completion, improve overall cost of computation, prevent any one resource from becoming the bottleneck, and meeting deadlines for all tasks. Good load balancing improves reliability, increases resource usage and ultimately improves user satisfaction.

A viable option for solving cloud scheduling problems is through LSQ (Least Square-based) task allocation algorithm. It is widely praised to be a straightforward and computationally less costly method. This method uses simple statistical approaches to

model the relationship between tasks and available resources and aims to find an optimal or suboptimal scheduling solution with a low computational burden. In the end it is trying to balance in an intelligent way the three tradeoffs: time, money, and energy consumption.

The LSQ-based scheduling algorithm can show strong improvements in performance when executed within a development environment like Visual Studio. The LSQ can be utilized in conjunction with simulation systems and cloud APIs, making it a practical platform to build and evaluate cloud workloads and scheduling solutions. Furthermore, its performance may be ensured by using it in parallel with decision-support systems or machine learning models that can adapt to real-time changes in both resource availability and workload patterns.

References

[1] S.s Manvi, G.k. Shyam, “resource management for infrastructure as service in cloud computing”, 2014.

[2] T. M. Thomas , T. Oliveira , “ Assessing the determinents of cloud computing adoption; ana analysis of the manufacturing and service sectors”, 2014.

[3] H. Y. Chong, J. S. Wong, and X. Wang, "An explanatory case study on cloud computing applications in the built environment," Automation in Construction, vol. 44, pp. 152-162, 2014.

[4] C. Gurkok, "Securing cloud computing systems," in Network and System Security (Second Edition), J. R. Vacca, Ed, 2014, Syngress: Boston, ch. 4, pp. 83-126.

[5] N. J. Navimipour and L. M. Khanli, "The LGR method for taskscheduling in computational grid," in Proc. International Conference on Advanced Computer Theory and Engineering , Phuket, 2008, pp. 1062-1066.

[6] F. Zhang et al., "Multi-objective scheduling of many tasks in cloudplatforms," Future Generation Computer Systems, vol. 37, pp. 309-320, 2014.

[7] J. O. G. Garcia and K. M. Sim, "A family of heuristics for agent-based elastic cloud bag-of-tasks concurrent scheduling," Future Generation Computer Systems, vol. 29, no. 7, pp. 1682-1699, 2013.

[8] X. S. Yang and S. Deb, "Cuckoo search via levy flights," in Proc. The NaBIC 2009, World Congress on Nature and Biologically InspiredComputing 2009, 2009, pp. 210-214.

[9] C. A. Santos, P. K. Freire, and S. K. Mishra, "Cuckoo search via levy flights for optimization of a physically-based runoff-erosion model," Journal of Urban and Environmental Engineering, vol. 6, no. 2, 2012.

[10] S. Pandey et al., "A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments," in Proc. 2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA), 2010.

[11] M. Mezmaz et al., "A parallel bi-objective hybrid metaheuristic for energy-aware scheduling for cloud computing systems," Journal of Parallel and Distributed Computing, vol. 71, no. 11, pp. 1497-1508, 2011.

[12] S. Abrishami and M. Naghibzadeh, "Deadline-constrained workflow scheduling in software as a service cloud," Scientia Iranica, vol. 19, no. 3, pp. 680-689, 2012.

[13] W. Wang et al., "Cloud-DLS: Dynamic trusted scheduling for Cloud computing," Expert Systems with Applications, vol. 39, no. 3, pp. 2321-2329, 2012.

[14] A. Dogan and F. Ozguner, "Matching and scheduling algorithms for minimizing execution time and failure probability of applications in heterogeneous computing," IEEE Transactions on Parallel and Distributed Systems, vol.13, no. 3, pp. 308-323, 2002.

[15] Y. Laili et al., "A Ranking Chaos Algorithm for dual scheduling of cloud service and computing resource in private cloud," Computers in Industry, vol. 64, no. 4, pp. 448-463, 2013.